

$$E = mc^2$$

$$\int f(x) dx = F(b) - F(a)$$

$$a^2 + b^2 = c^2$$

$$e^{i\pi} + 1 = 0$$

$$\lim_{x \rightarrow \infty} \frac{1}{x} = 0$$

Matemáticas para IA

S2.1 · FUNDAMENTOS MATEMÁTICOS

Un recorrido esencial por el álgebra lineal, el cálculo y la optimización que hace posible el aprendizaje automático moderno. Desde vectores hasta descenso del gradiente, aquí están las herramientas que todo practicante de IA necesita dominar.

Contenidos de la sesión

01

Vectores y matrices

Representación de datos y transformaciones lineales

02

Operaciones fundamentales

Producto escalar, matricial, transpuesta e inversa

03

Autovalores y autovectores

Intuición geométrica y aplicaciones

04

Funciones y derivadas

Del cálculo univariable al gradiente multidimensional

05

Optimización

Función de coste y descenso del gradiente

¿Por qué las matemáticas en IA?

La inteligencia artificial no es magia: es matemática aplicada a escala. Los algoritmos de aprendizaje automático son, en esencia, operaciones algebraicas y de optimización repetidas millones de veces sobre datos numéricos.



El lenguaje secreto de la IA



Datos como vectores

Una imagen, un texto o un sonido se representan como vectores numéricos. El álgebra lineal define cómo manipular estas representaciones de forma eficiente.



Aprendizaje como optimización

Entrenar un modelo es minimizar una función de error. El cálculo diferencial nos da la dirección de descenso hacia el mínimo óptimo.



Probabilidad y estadística

Los modelos aprenden distribuciones de datos. La probabilidad cuantifica la incertidumbre inherente en toda predicción.

Vectores

El bloque fundamental sobre el que se construye toda la IA

¿Qué es un vector?

Un **vector** es una lista ordenada de números que representa una dirección y magnitud en el espacio. En IA, un vector es la forma natural de representar cualquier dato: una muestra de audio, los píxeles de una imagen o las características de un usuario.



Vectores: representación y notación

Notación matemática

Un vector en $\mathbb{R}^n$ se escribe como una columna de n componentes reales:

$$\vec{v} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

Ejemplo en Python / NumPy

```
import numpy as np

# Vector en R^3
v = np.array([1.0, 2.5, -0.7])

# Norma (longitud)
norma = np.linalg.norm(v)
print(norma) # 2.70
```

¿Qué es una matriz?

Una **matriz** es una tabla bidimensional de números organizados en filas y columnas. Las matrices son la herramienta para representar transformaciones lineales, conjuntos de datos (donde cada fila es una muestra) o los pesos de una red neuronal.

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \in \mathbb{R}^{2 \times 3}$$

Filas

Muestras de datos o ecuaciones del sistema

Columnas

Características o variables del problema

Dimensión

Se escribe como **m × n** (filas × columnas)

Operaciones

Producto escalar, producto matricial, transpuesta e inversa

Producto escalar (dot product)

Definición

El producto escalar de dos vectores $\mathbf{u}$ y $\mathbf{v}$ de la misma dimensión es la suma de los productos de sus componentes:

$$\vec{u} \cdot \vec{v} = \sum_{i=1}^n u_i v_i = |\vec{u}| |\vec{v}| \cos \theta$$

El resultado es un **escalar** (un único número). Mide cuánto se «alinean» dos vectores.

En IA, sirve para...

- Calcular similitud entre vectores de características
- Aplicar pesos en una neurona artificial:
 $z = \vec{w} \cdot \vec{x} + b$
- Medir la similitud del coseno en búsqueda semántica

Producto matricial

La operación más central del deep learning. Para multiplicar **A** (**m**×**k**) por **B** (**k**×**n**), cada elemento de la matriz resultante **C** (**m**×**n**) es el producto escalar de una fila de A con una columna de B:

$$C_{ij} = \sum_{r=1}^k A_{ir} \cdot B_{rj}$$

- ❏ Regla de dimensiones: el número de columnas de A debe coincidir con el número de filas de B. El resultado tiene las filas de A y las columnas de B.

Una capa densa de una red neuronal aplica exactamente: $\mathbf{y} = A\mathbf{x} + \mathbf{b}$, donde A es la matriz de pesos y x el vector de entrada.

Transpuesta e inversa

Transpuesta A^T

Intercambia filas y columnas. Si A es $m \times n$, entonces A^T es $n \times m$.

Propiedad clave: $(AB)^T = B^T A^T$.

Se usa constantemente en backpropagation.

Inversa A^{-1}

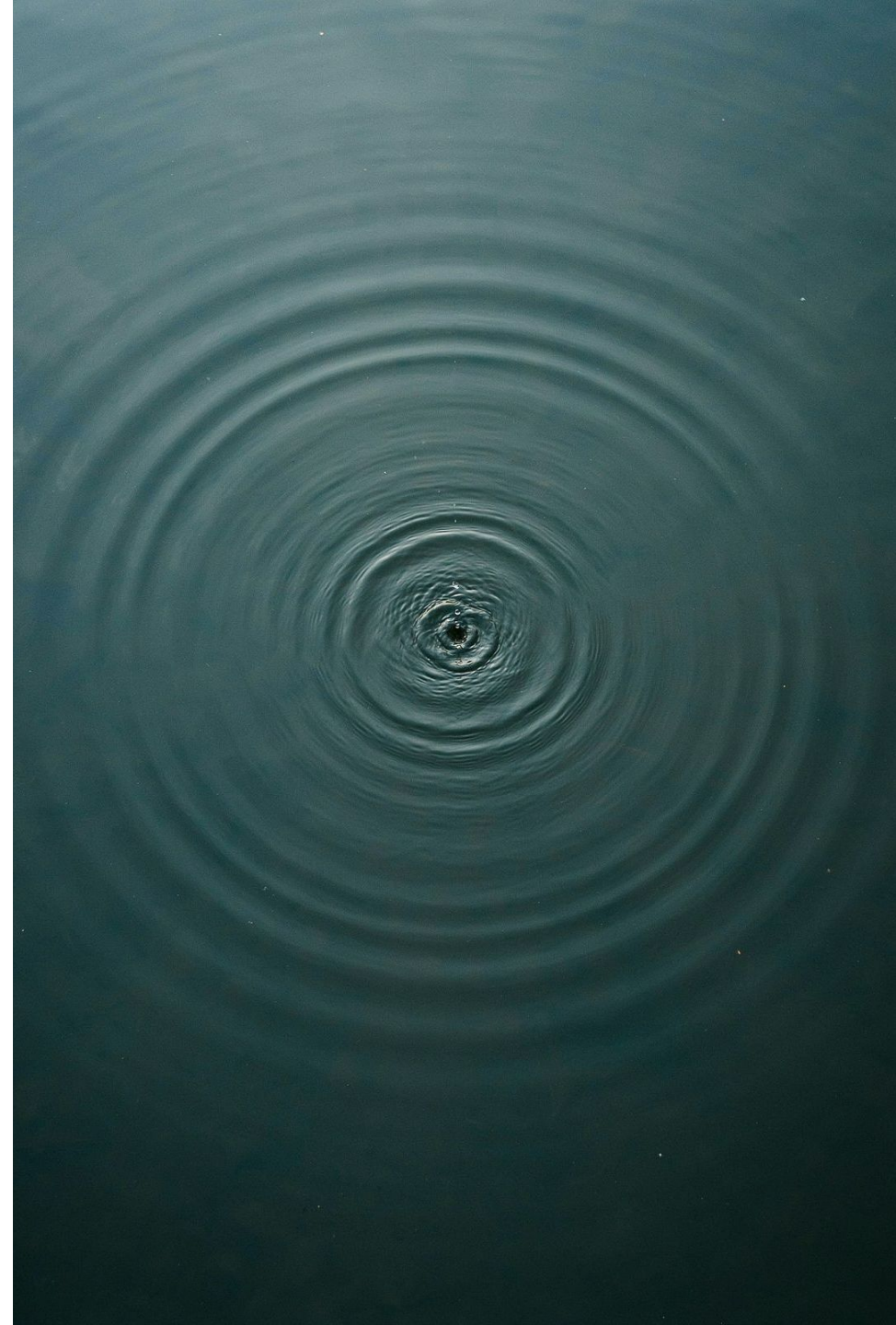
Solo existe para matrices cuadradas no singulares. Cumple $AA^{-1} = I$. Permite «deshacer» la transformación lineal. En la práctica, se evita calcularla directamente por coste computacional.

Pseudoinversa A^+

Generalización de la inversa para matrices no cuadradas. La solución de mínimos cuadrados es $\hat{x} = A^+b$, fundamental en regresión lineal.

Autovalores y Autovectores

Una idea geométrica poderosa con aplicaciones en PCA, compresión y estabilidad de redes neuronales



La idea intuitiva

Definición

Sea A una matriz cuadrada. Un vector $\mathbf{v} \neq \mathbf{0}$ es un **autovector** de A si al multiplicarlo solo cambia de escala, no de dirección:

$$A\vec{v} = \lambda\vec{v}$$

El escalar λ (lambda) se llama **autovalor** asociado a $\mathbf{v}$.

Aplicaciones en IA

- **PCA**: los autovectores de la matriz de covarianza definen las direcciones de máxima varianza
- **PageRank**: la página web más relevante es el autovector dominante del grafo de la web
- **Redes recurrentes**: los autovalores determinan la estabilidad del entrenamiento

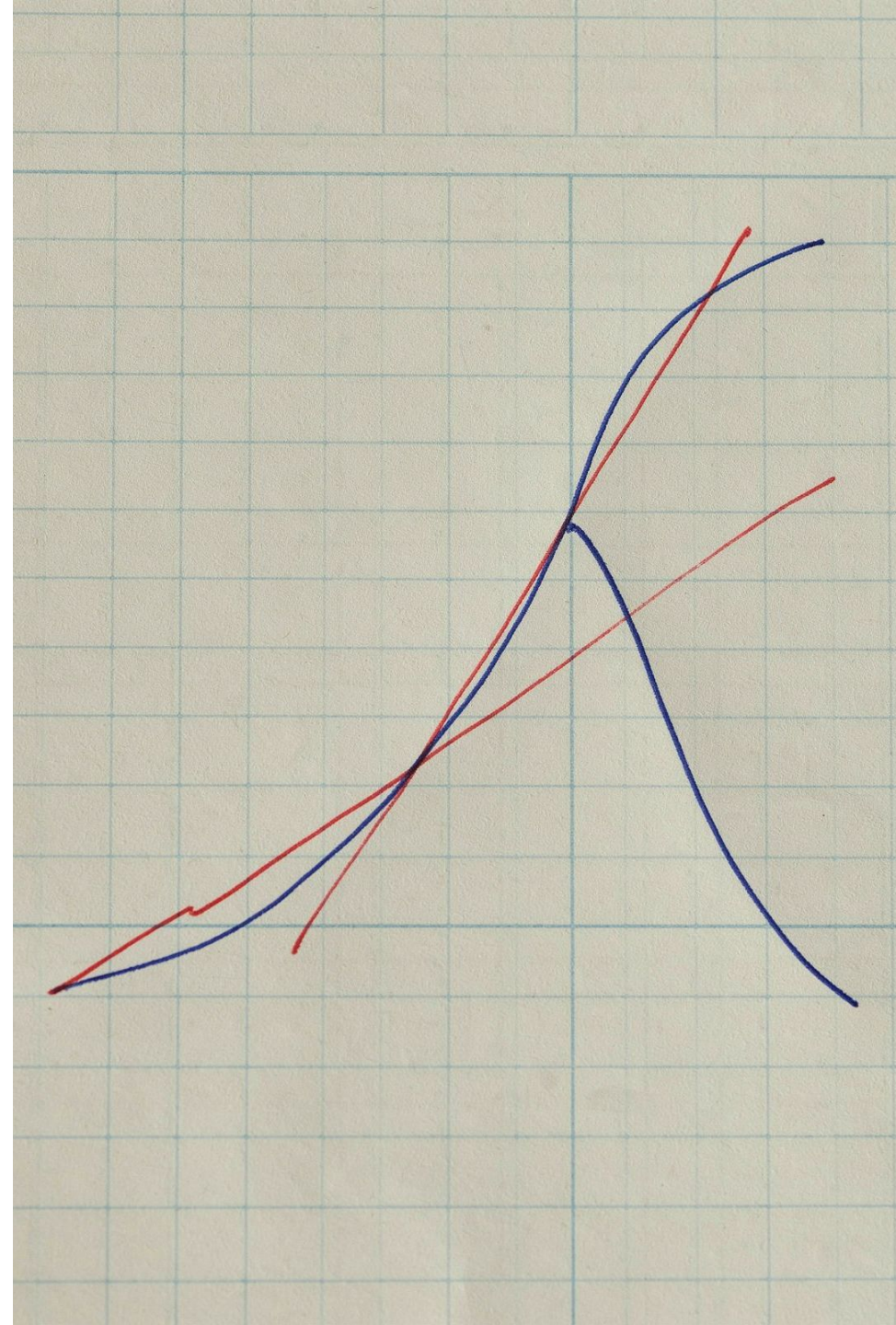
CÁLCULO

Funciones y Derivadas

El motor matemático del aprendizaje

Funciones: de la entrada a la salida

Una **función** $f : \mathbb{R} \rightarrow \mathbb{R}$ asigna a cada valor de entrada un único valor de salida. En IA, las funciones de activación como ReLU, sigmoid o tanh transforman señales en cada neurona y determinan si esta «se activa» o no.



La derivada: tasa de cambio instantáneo

Definición formal

La derivada de f en el punto x mide cuánto cambia f ante una variación infinitesimal de x :

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$$

Geométricamente, es la **pendiente de la recta tangente** a la curva en ese punto.

Derivadas comunes en IA

Función	Derivada
x^n	nx^{n-1}
e^x	e^x
$\ln(x)$	$1/x$
$\sigma(x)$	$\sigma(x)(1 - \sigma(x))$

Derivadas Parciales y Gradiente

Cuando las funciones dependen de varias variables —como ocurre siempre en IA— necesitamos derivar respecto a cada una por separado

Derivada parcial: un paso a la vez

La **derivada parcial** de $f(x_1, x_2, \dots, x_n)$ respecto a x_i mide cómo cambia f cuando solo varía x_i , manteniendo las demás constantes. Se denota $\frac{\partial f}{\partial x_i}$.

Ejemplo

$$f(x, y) = 3x^2 + 2xy + y^3$$

$$\frac{\partial f}{\partial x} = 6x + 2y$$

$$\frac{\partial f}{\partial y} = 2x + 3y^2$$

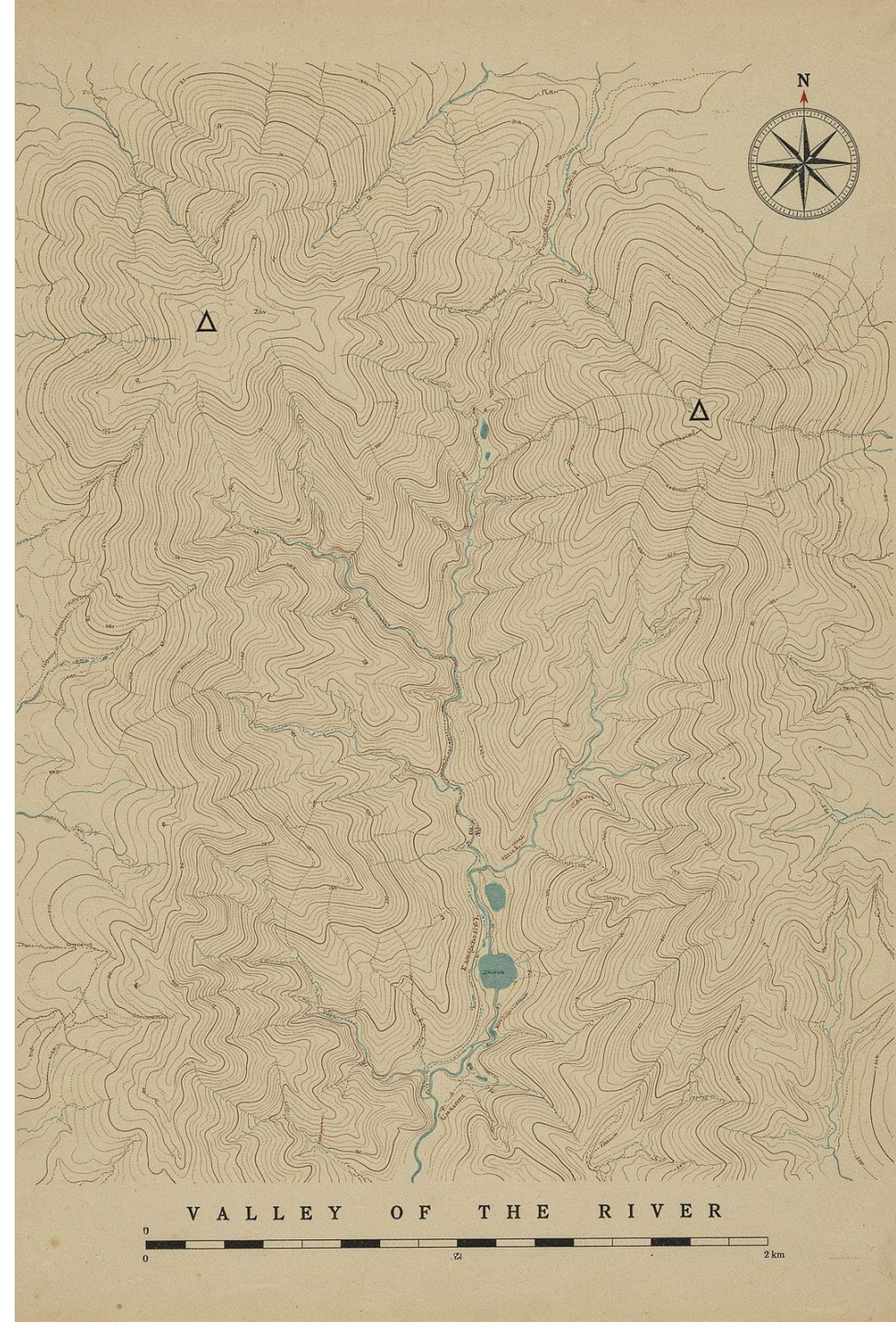
Intuición

Imagina que estás en una montaña. La derivada parcial respecto a x te dice cuánto subes o bajas si te mueves únicamente hacia el Este, ignorando el Norte. La derivada respecto a y hace lo mismo pero en dirección Norte.

El gradiente: la brújula del aprendizaje

El **gradiente** agrupa todas las derivadas parciales en un solo vector. Apunta siempre en la dirección de **máximo crecimiento** de la función. En optimización, nos moveremos en la dirección *opuesta* para minimizar el error:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$



CÁLCULO

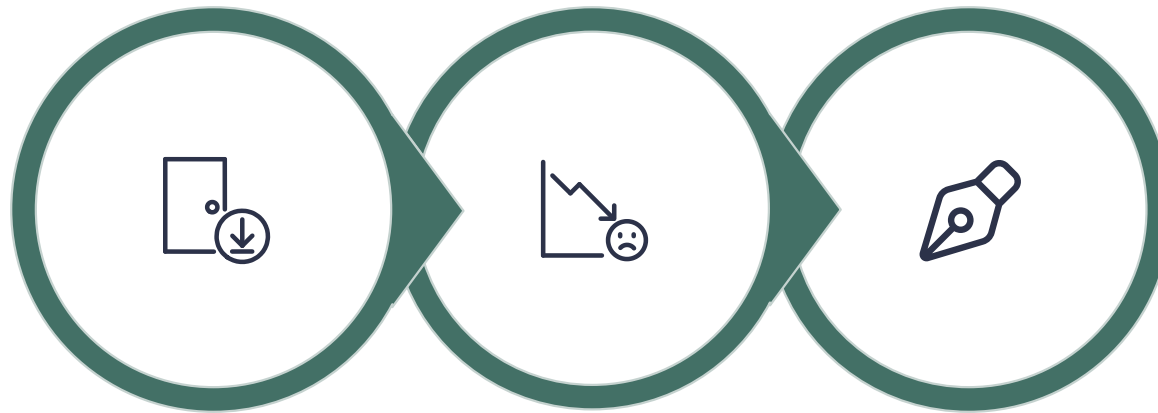
Regla de la Cadena

El corazón matemático de la retropropagación en redes neuronales

Regla de la cadena: composición de funciones

Cuando una función se compone de varias funciones anidadas — exactamente como ocurre en las capas de una red neuronal — la regla de la cadena nos permite derivar el todo a partir de las partes:

$$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$$



Entrada

Salida

Retroprop.

Regla de la cadena: ejemplo concreto

Composición de tres funciones

Sea $L = f(g(h(x)))$. La derivada total se encadena multiplicando cada derivada local:

$$\frac{dL}{dx} = \frac{dL}{df} \cdot \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}$$

Backpropagation en 3 capas

En una red neuronal, cada capa aplica una transformación. Durante el entrenamiento, el error calculado en la salida debe «viajar hacia atrás» a través de todas las capas para ajustar cada peso:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial a_2} \cdot \frac{\partial a_2}{\partial w_1}$$

- ❏ La regla de la cadena aplicada recursivamente es exactamente el algoritmo de **backpropagation**, el pilar del entrenamiento de redes neuronales profundas.

Función de Coste

¿Cómo medimos cuánto se equivoca un modelo?



¿Qué es la función de coste?

La **función de coste** (también llamada función de pérdida o *loss function*) cuantifica numéricamente la diferencia entre las predicciones del modelo y los valores reales. Cuanto mayor es el coste, peor es el modelo. Entrenar consiste en encontrar los parámetros que **minimizan** esta función.

Funciones de coste habituales

Error Cuadrático Medio (MSE)

Para regresión. Penaliza los errores grandes de forma cuadrática:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Entropía cruzada binaria

Para clasificación binaria. Mide la divergencia entre distribuciones de probabilidad:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Entropía cruzada categórica

Para clasificación multiclase con softmax. Generaliza la binaria a K clases:

$$L = -\sum_{k=1}^K y_k \log \hat{p}_k$$

Descenso del Gradiente

El algoritmo que «baja la montaña» buscando el mínimo de la función de coste

Cómo funciona el descenso del gradiente

La regla de actualización

En cada iteración, los parámetros se mueven una pequeña cantidad en la dirección opuesta al gradiente:

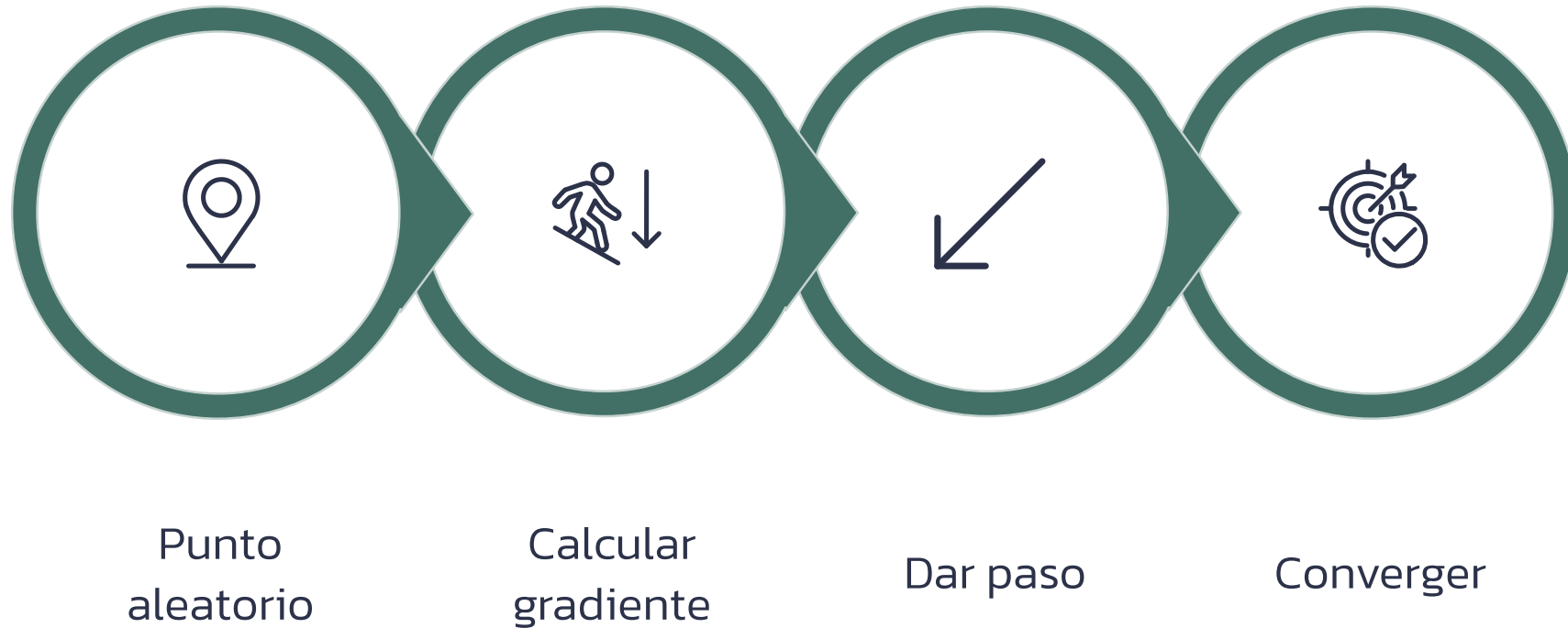
$$\theta \leftarrow \theta - \alpha \cdot \nabla_{\theta} L(\theta)$$

Donde α es la **tasa de aprendizaje** (*learning rate*), un hiperparámetro crítico.

Paso a paso

1. Inicializar los pesos θ aleatoriamente
2. Calcular el coste $L(\theta)$ con los datos actuales
3. Calcular el gradiente $\nabla_{\theta} L$ via backprop
4. Actualizar θ con la regla anterior
5. Repetir hasta convergencia

Visualizando el descenso del gradiente



La tasa de aprendizaje α controla el tamaño del paso: demasiado pequeña y la convergencia es lenta; demasiado grande y el algoritmo puede oscilar o divergir, saltando por encima del mínimo.

Variantes del descenso del gradiente

Batch GD

Usa **todos** los datos en cada paso. Gradiente exacto pero muy lento para datasets grandes. Garantiza convergencia suave.

Stochastic GD (SGD)

Usa **una sola muestra** por paso. Muy rápido pero ruidoso. El ruido puede ayudar a escapar de mínimos locales.

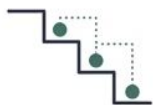
Mini-batch GD

Usa **pequeños lotes** (32-256 muestras). El equilibrio perfecto: eficiente en GPU y estable. Es el estándar en deep learning.

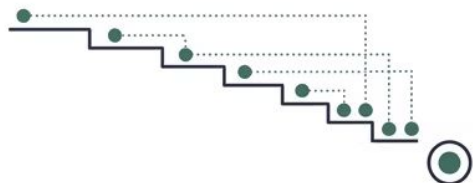
Adam / RMSProp

Tasas de aprendizaje adaptativas por parámetro. Adam combina momento y escala del gradiente. Es el optimizador más usado hoy.

El efecto de la tasa de aprendizaje



Demasiado Pequeño
($\alpha=0.001$)



Muchas iteraciones,
convergencia lenta,
desperdicia cómputo.



Justo
($\alpha=0.01$)
($\alpha=0.01$)



Descenso suave,
alcanza el mínimo
de forma eficiente.



Demasiado Grande
($\alpha=0.5$)



Oscila violentamente,
puede divergir,
nunca converge.

Encontrar la tasa de aprendizaje adecuada es uno de los ajustes más importantes en el entrenamiento. Técnicas como el **learning rate scheduler** y el **warm-up** permiten variarlo dinámicamente durante el entrenamiento.

PRÁCTICA

Ejercicios con Desmos y GeoGebra

Visualizar para comprender profundamente



Herramientas de visualización matemática

Las matemáticas abstractas cobran vida cuando las vemos en acción. **Desmos** y **GeoGebra** son calculadoras gráficas interactivas gratuitas, accesibles desde el navegador, que permiten explorar funciones, derivadas, vectores y superficies 3D sin necesidad de instalar nada.

Ejercicios propuestos con Desmos

1 Explora la derivada gráficamente

Dibuja $f(x) = x^3 - 3x$ y activa su derivada $f'(x)$. Observa dónde la pendiente es cero y cómo se relaciona con los máximos y mínimos de f .

2 Simula descenso del gradiente

Usa un deslizador para α y visualiza cómo el punto $(x, f(x))$ se desplaza iteración a iteración hacia el mínimo de $f(x) = x^2 + 2$.

3 Regla de la cadena visual

Compone dos funciones $g(x) = \sin(x)$ y $h(x) = 2x + 1$, forma $f = g(h(x))$ y comprueba que su derivada coincide con la fórmula de la cadena.

Ejercicios propuestos con GeoGebra

1

Vectores en 2D y 3D

Crea dos vectores $\vec{u}$ y $\vec{v}$ en GeoGebra 3D. Calcula su producto escalar y observa cómo varía el ángulo entre ellos al arrastrar los puntos.

2

Transformación matricial

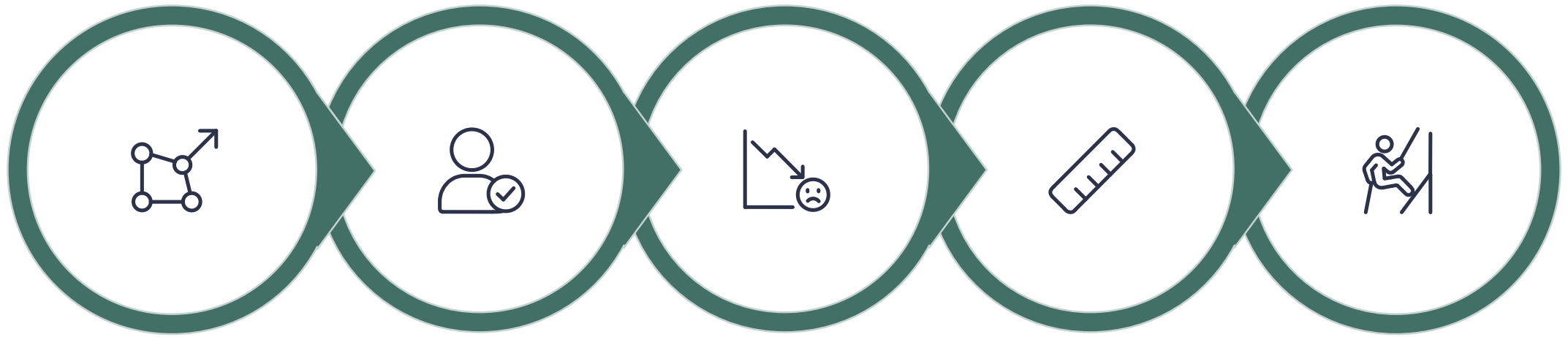
Define una matriz 2×2 y aplícala a un conjunto de vectores. Visualiza cómo la matriz rota, escala o refleja el plano. Modifica los coeficientes y observa el efecto en tiempo real.

3

Superficie 3D y gradiente

Grafica $f(x, y) = x^2 + y^2$ como superficie 3D. Añade el vector gradiente en un punto y comprueba que siempre apunta hacia la dirección de mayor pendiente ascendente.

Conectando todo: de los datos al modelo



Datos

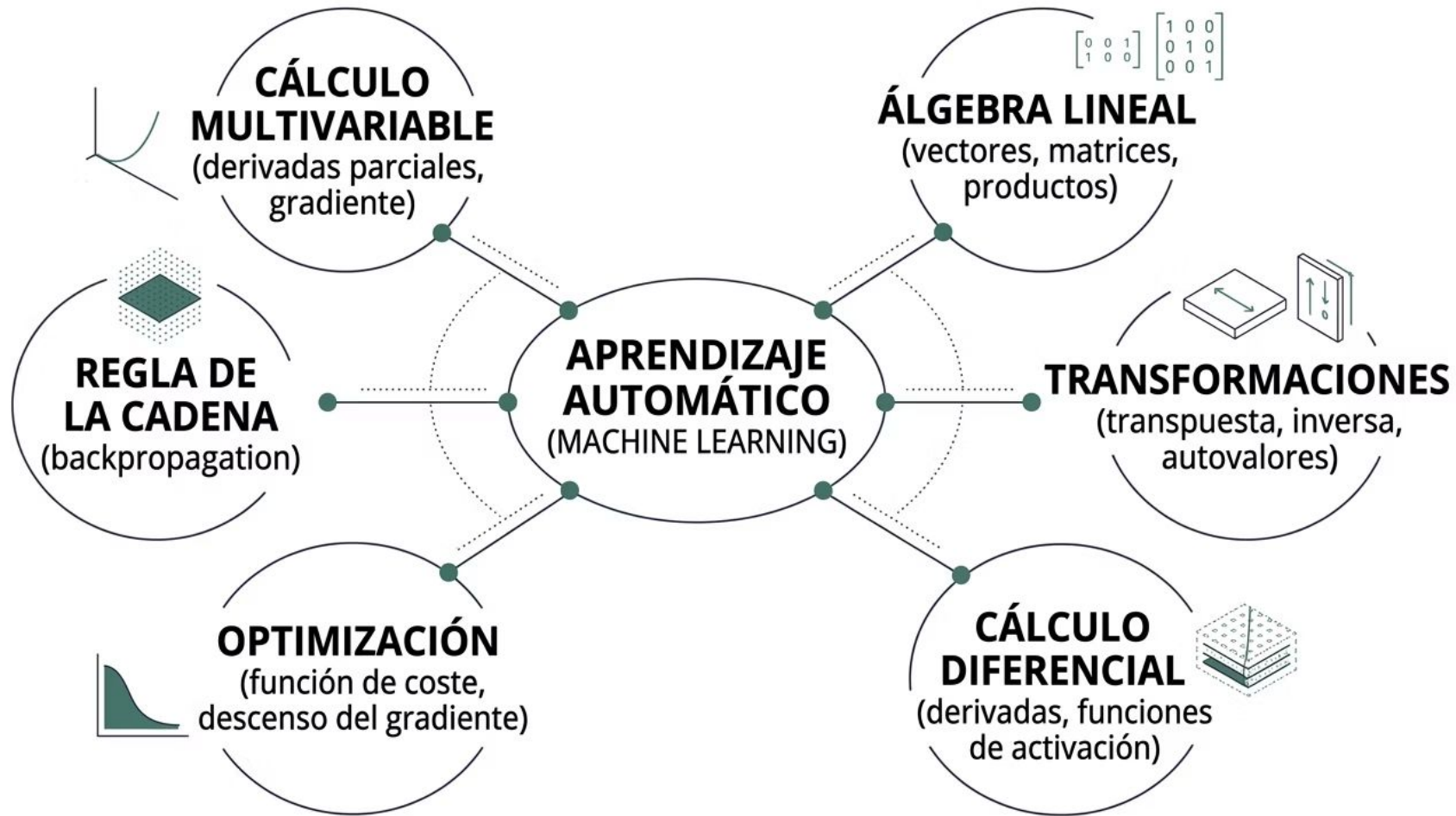
Propagación

Pérdida

Retroprop.

Actualiza

Resumen: el mapa matemático de la IA



Conceptos clave de la sesión



Vectores y matrices

Representación universal de datos y transformaciones en IA



Gradiente

El vector que señala la dirección de máximo crecimiento (o caída) de cualquier función



Regla de la cadena

Fundamento matemático del algoritmo de backpropagation en redes profundas



Descenso del gradiente

Algoritmo iterativo que minimiza la función de coste ajustando los pesos del modelo

Recursos para profundizar

→ 3Blue1Brown — Essence of Linear Algebra

Serie de vídeos animados en YouTube que visualiza la álgebra lineal de forma intuitiva e insuperable. Imprescindible para todo estudiante de IA.

→ Khan Academy — Cálculo multivariable

Curso gratuito y en español con ejercicios interactivos sobre derivadas parciales, gradiente y optimización, ideales para afianzar la base.

→ Mathematics for Machine Learning (Deisenroth et al.)

Libro de referencia disponible gratuitamente en PDF. Cubre álgebra lineal, probabilidad y optimización con el enfoque específico del aprendizaje automático.

Próximos pasos

Antes de la próxima sesión

- Completa los tres ejercicios de Desmos y GeoGebra
- Implementa descenso del gradiente en Python sobre MSE
- Lee las páginas 1–40 del libro de Deisenroth et al.

En la próxima sesión (S2.2)

- Probabilidad y estadística bayesiana
- Distribuciones, esperanza y varianza
- Máxima verosimilitud y estimación MAP

